



東南大學
SOUTHEAST UNIVERSITY

OPERATING SYSTEM CONCEPTS

.....

Exercises

A/Prof. Kai Dong



Contents

1. Chapter 1. Introduction
2. Chapter 2. Operating System Structures
3. Chapter 3. Processes
4. Chapter 4. Threads
5. Chapter 5. CPU Scheduling
6. Chapter 6. Process Synchronization

Exercises 1.19



What is the purpose of interrupts? How does an interrupt differ from a trap? Can traps be generated intentionally by a user program? If so, for what purpose?

中断和陷阱都是操作系统用来处理特定事件的机制，但它们在目的和使用方式上有所不同。

1. 中断的目的：

- 中断 (Interrupt) 主要用于处理外部事件，如硬件设备的信号。它允许操作系统响应外部事件（比如键盘输入、网络数据到达等），确保系统能及时处理这些事件。

- 中断是异步的，通常由硬件设备触发，不由程序直接控制。

2. - 陷阱 (Trap) 是一种特殊类型的中断，通常用于内部事件，如软件错误或系统调用。

- 陷阱可以是同步的，由执行中的程序指令显式触发。例如，当程序进行系统调用请求操作系统服务（如文件操作、进程管理等）时，会使用陷阱。

- 与中断相比，陷阱通常由程序执行的指令显式触发，而不是由外部硬件事件触发。

3. - 是的，用户程序可以有意生成陷阱。这主要通过执行特定的指令（如 `int` 指令在 x86 架构上）来完成。

- 生成陷阱的主要目的是进行系统调用。系统调用是用户程序请求操作系统提供服务的一种方式，比如读写文件、创建或终止进程、请求更多内存等。通过系统调用，程序可以执行它无法直接进行的操作，因为这些操作需要更高的权限。

总结来说，中断和陷阱都是操作系统用来响应事件的机制，它们帮助操作系统在控制和数据流中介入，确保系统的稳定性和效率。陷阱的使用尤其对于系统调用的实现至关重要，使得用户程序能够安全地请求和执行更复杂的操作。



Contents

1. Chapter 1. Introduction
- 2. Chapter 2. Operating System Structures**
3. Chapter 3. Processes
4. Chapter 4. Threads
5. Chapter 5. CPU Scheduling
6. Chapter 6. Process Synchronization

Exercises 2.1



What is the purpose of system calls?

Exercises 2.13



Describe three general methods for passing parameters to the operating system.

系统调用是操作系统提供了一种机制，允许用户空间的程序请求操作系统核心提供服务，它是用户程序与操作系统之间的接口。系统调用的主要目的包括：

1. ****资源管理**：**

- 操作系统需要管理计算机的硬件资源，如处理器、内存、磁盘和网络设备。系统调用允许用户程序请求操作系统进行资源的分配、管理和释放，如打开文件、分配内存等。

2. ****抽象化硬件**：**

- 系统调用为硬件设备提供了一个抽象层，使得程序员不需要对具体的硬件细节有深入了解即可进行编程。例如，文件系统的系统调用允许用户读写文件，而无需关心磁盘的物理或逻辑结构。

3. ****安全和权限控制**：**

- 系统调用使得操作系统能够在用户程序和硬件资源之间施加访问控制，确保用户程序在不超越其权限的情况下运行。例如，某个程序试图访问它没有权限的文件时，操作系统可以通过系统调用来拒绝这种访问。

4. ****进程控制**：**

- 系统调用可以用来控制其他进程，例如创建新进程、结束进程、设置进程属性等。这是多任务操作系统中非常核心的功能。

5. ****通信**：**

- 系统调用还包括进程间通信（IPC）机制，如信号量、消息队列和共享内存，这些都是通过系统调用来实现的。这使得运行在系统上的不同程序能够协同工作。

总体来说，系统调用为用户程序提供了一个有效且安全的方式来执行那些需要更高权限的操作，同时也让操作系统能够控制对其资源的访问和管理，从而保护系统的稳定性和安全性。

2.3 讨论向操作系统传递参数的三个主要的方法。

Answer：

1. 通过寄存器来传递参数

2. 寄存器传递参数块的首地址

3. 参数通过程序存放或压进堆栈中，并通过操作系统弹出堆栈。



Contents

1. Chapter 1. Introduction
2. Chapter 2. Operating System Structures
- 3. Chapter 3. Processes**
4. Chapter 4. Threads
5. Chapter 5. CPU Scheduling
6. Chapter 6. Process Synchronization



Exercises 3.5

When a process creates a new process using the `fork()` operation, which of the following states is shared between the parent process and the child process?

1. Stack
2. Heap
3. Shared memory segments

在 `fork()` 后，栈和堆是不共享的，它们在父子进程间是独立的副本；而共享内存段则是共享的，允许父子进程间进行数据共享。



Exercises 3.8

Describe the differences among short-term, medium-term, and long-term scheduling.

3.1 论述短期, 中期和长期调度之间的区别.

Answer: a. 短期调度: 在内存作业中选择就绪执行的作业, 并为他们分配 CPU。

b. 中期调度: 作为一种中等程度的调度程序, 尤其被用于分时系统, 一个交换方案的实施, 将部分运行程序移出内存, 之后, 从中断处继续执行。

c. 长期调度 (作业调度程序): 确定哪些作业调入内存以执行。

它们主要的不同之处是它们的执行的频率。短期调度必须经常调用一个新进程, 由于在系统中, 长期调度处理移动的作业时, 并不频繁被调用, 可能在进程离开系统时才被唤起。



Contents

1. Chapter 1. Introduction
2. Chapter 2. Operating System Structures
3. Chapter 3. Processes
- 4. Chapter 4. Threads**
5. Chapter 5. CPU Scheduling
6. Chapter 6. Process Synchronization

Exercises 4.2



What are two differences between user-level threads and kernel-level threads? Under what circumstances is one type better than the other?

用户级线程 (User-level threads) 和内核级线程 (Kernel-level threads) 之间有几个关键的区别，主要涉及管理方式和操作系统的支持。了解这些区别有助于在特定情况下选择更适合的线程类型：

1. ****管理****：

- ****用户级线程****：这些线程完全在用户空间中管理，不需要操作系统的直接干预。线程的创建、同步、调度和管理都是通过用户空间的库（如 POSIX 线程库）来完成的。这种方式的好处是速度快，因为不需要系统调用；缺点是，操作系统不知道这些线程的存在，无法进行智能调度。

- ****内核级线程****：由操作系统内核直接管理和调度。内核有完整的线程信息，可以更有效地管理 CPU 资源和线程调度。这种方式的优点是可以利用多核处理器的并发性，缺点是创建和管理线程时需要系统调用，比用户级线程开销大。

2. ****性能和资源利用****：

- ****用户级线程****：由于不需要与内核交互，执行系统调用的开销更小。然而，如果一个用户级线程进行了阻塞操作（如 I/O 请求），它会导致整个进程的所有线程都被阻塞。

- ****内核级线程****：由于内核控制线程，一个线程的阻塞操作不会影响其他线程。这使得内核级线程能更好地处理多线程的并发和资源共享。

****不同情况下的适用性****：

- ****用户级线程更适合的情况****：当应用程序进行大量的计算而很少进行阻塞操作（如 I/O）时，用户级线程可能更优。这种情况下，线程管理的开销小，且线程间的切换速度快。

- ****内核级线程更适合的情况****：在需要大量的并发执行和利用多核处理器能力的应用中，内核级线程更为合适。特别是当线程需要执行阻塞性操作时，使用内核级线程可以避免一个线程的阻塞影响到整个程序。

因此，选择哪种线程类型取决于应用程序的特定需求和预期的线程行为。内核级线程虽然在创建和管理上的开销较大，但在多核环境下的表现以及对阻塞性操作的处理上更加优越。用户级线程在轻量级任务和计算密集型应用中可能更加高效。



Exercises 4.8

Which of the following components of program state are shared across threads in a multithreaded process?

1. Register values
2. Heap memory
3. Global variables
4. Stack memory

4.4 以下程序中的哪些组成部分在多线程程序中是被线程共享的？

- a. 寄存器值
- b. 堆内存
- c. 全局变量
- d. 栈内存

答：一个线程程序的线程共享堆内存和全局变量，但每个线程都有属于自己的一组寄存器和栈内存。



Contents

1. Chapter 1. Introduction
2. Chapter 2. Operating System Structures
3. Chapter 3. Processes
4. Chapter 4. Threads
- 5. Chapter 5. CPU Scheduling**
6. Chapter 6. Process Synchronization



Exercises 6.3

Suppose that the following processes arrive for execution at the times indicated. Each process will run for the amount of time listed. In answering the questions, use nonpreemptive scheduling, and base all decisions on the information you have at the time the decision must be made.

Process	Arrival Time	Burst Time
P1	0.0	8
P2	0.4	4
P3	1.0	1

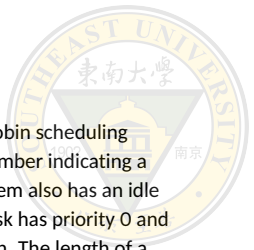
1. What is the average turnaround time for these processes with the FCFS scheduling algorithm?
2. What is the average turnaround time for these processes with the SJF scheduling algorithm?
3. The SJF algorithm is supposed to improve performance, but notice that we chose to run process P1 at time 0 because we did not know that two shorter processes would arrive soon. Compute what the average turnaround time will be if the CPU is left idle for the first 1 unit and then SJF scheduling is used. Remember that processes P1 and P2 are waiting during this idle time, so their waiting time may increase. This algorithm could be called future-knowledge scheduling.



Key to 6.3

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
FCFS:	P_1								P_2			P_3		
SJF:	P_1								P_3	P_2				
SJF':		P_3	P_2				P_1							

1. $T_{\text{turnaround}} = \frac{8 + (12 - 0.4) + (13 - 1)}{3} \approx 10.53$
2. $T_{\text{turnaround}} = \frac{8 + (13 - 0.4) + (9 - 1)}{3} \approx 9.53$
3. $T_{\text{turnaround}} = \frac{14 + (6 - 0.4) + (2 - 1)}{3} \approx 6.87$



Exercises 6.17

The following processes are being scheduled using a preemptive, roundrobin scheduling algorithm. Each process is assigned a numerical priority, with a higher number indicating a higher relative priority. In addition to the processes listed below, the system also has an idle task (which consumes no CPU resources and is identified as P_{idle}). This task has priority 0 and is scheduled whenever the system has no other available processes to run. The length of a time quantum is 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

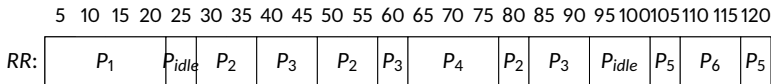
Thread	Priority	Burst	Arrival
P1	40	20	0
P2	30	25	25
P3	30	25	30
P4	35	15	60
P5	5	10	100
P6	10	10	105

1. Show the scheduling order of the processes using a Gantt chart.
2. What is the turnaround time for each process?
3. What is the waiting time for each process?
4. What is the CPU utilization rate?

Key to 6.17



1. Gantt chart:



2. $T_{turnaround} = T_{completion} - T_{arrival}$

3. $T_{waiting} = T_{turnaround} - T_{burst}$

	P_1	P_2	P_3	P_4	P_5	P_6
$T_{turnaround}$	20-0=20	80-25=55	90-30=60	75-60=15	120-100=20	115-105=10
$T_{waiting}$	20-20=0	55-25=30	60-25=35	15-15=0	20-10=10	10-10=0

4. $rate = \frac{105}{120} \approx 87.50\%$



Exercises 6.19

Which of the following scheduling algorithms could result in starvation?

1. First-come, first-served
2. Shortest job first
3. Round robin
4. Priority

最短作业优先（第2个）和优先级（第4个）调度算法可能导致饿死



Contents

1. Chapter 1. Introduction
2. Chapter 2. Operating System Structures
3. Chapter 3. Processes
4. Chapter 4. Threads
5. Chapter 5. CPU Scheduling
- 6. Chapter 6. Process Synchronization**



Exercises 5.3

What is the meaning of the term busy waiting? What other kinds of waiting are there in an operating system? Can busy waiting be avoided altogether? Explain your answer.

6.3 忙等待的含义是什么？在操作系统中还有哪些其他形式的等待？忙等待能完全避免吗？给出你的答案。

答：忙等待意味着一个进程正在等待满足一个没有闲置处理器的严格循环的条件。或者，一个进程通过放弃处理器来等待，在这种情况下的块等待在将来某个适当的时间被唤醒。忙等待能够避免，但是承担这种开销与让一个进程处于沉睡状态，当相应程序的状态达到的时候进程又被唤醒有关。

忙等待 (busy waiting) 通常指的是进程或线程在等待某个条件变为真时，反复检查这个条件是否满足，而在检查过程中不释放CPU资源。这意味着该进程或线程会持续占用处理器，进行无休止的循环 (spin loop)，直到其等待的条件被满足。

忙等待的主要缺点是它消耗大量的处理器资源，因为它不断地在循环中执行，即使没有实际的工作完成。这会导致CPU使用率不必要地高，影响系统的整体性能。

****操作系统中的其他等待形式**：**

1. ****阻塞等待 (Block waiting) **：**

- 在阻塞等待中，如果进程等待某个条件（如I/O操作完成、获取信号量等），它会进入休眠状态，并释放CPU供其他进程使用。这种方式更高效，因为它允许操作系统调度其他可能可以运行的进程。

2. ****有条件的等待 (Conditional waiting) **：**

- 这涉及到进程或线程等待特定条件的满足，通常与同步机制（如条件变量）相关。在这种方式中，进程等待某个特定的条件发生，一旦条件满足，它会被操作系统唤醒。

忙等待**是否能完全避免**的问题：

- 忙等待在某些情况下是有用的，特别是在低延迟的同步操作中，或者在等待时间非常短的场景下。例如，对于多处理器系统中的细粒度锁定控制，忙等待可能因为避免了上下文切换的开销而更有效。
- 然而，在多数情况下，使用阻塞等待或有条件的等待是更好的选择，因为这些方法可以有效地利用处理器资源，减少不必要的CPU占用。
- 完全避免忙等待可能不是总是可行或理想的，特别是在某些特定的系统级别或低层次硬件交互的实现中，但通过设计更合理的同步策略和使用更高效的等待机制，可以最小化其使用。

总的来说，根据不同的应用场景和性能要求选择合适的等待策略是关键，避免过度使用忙等待以提高系统的整体效率和性能。



Exercises 5.4

Explain why spinlocks are not appropriate for single-processor systems yet are often used in multiprocessor systems.

答：自旋锁不适合在单处理器系统是因为从自旋锁中打破一个进程的条件只有在执行一个不同的进程时才能获得。如果这个进程没有闲置处理器，其他进程不能够得到这个机会去设定一个第一个进程进展需要的程序条件。在一个多处理器系统中，其他进程在其他处理器上执行，从而为了让第一个进程从自旋锁中释放修改程序状态。